UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

ESCUELA DE POSGRADO



CURSO: ESTRUCTURA DE DATOS Y ALGORITMOS

PROYECTO FINAL

PROFESOR: Heider Sanchez Enriquez

INTEGRANTES:

1. Axcel Daniel Fuentes Pacheco
2. Carlos Kiyoshi Fukumoto Chávez
3. Sebastián Tadeo Gallegos Mostajo

2026 -2

Informe del proyecto: pipeline de streams de comercio electrónico

El proyecto implementa un pipeline de 10 etapas que procesa en orden de llegada los eventos de una tienda en línea (dataset REES46) y termina en recomendaciones, alertas de fraude y consultas sobre el catálogo. Cada etapa usa una estructura de datos implementada en Python y la compara con su equivalente de la biblioteca estándar o con un método exacto. Las mediciones de 1K a 1M elementos confirman la complejidad esperada de cada estructura, y el pipeline completo funciona tanto con el stream sintético como con datos reales de REES46.

1. Contexto y arquitectura

REES46 registra vistas, carritos y compras con su hora, producto, categoría, marca, precio, usuario y sesión (mkechinov, s. f.). El archivo de octubre de 2019 tiene unos 42 millones de eventos en 5 GB. Como no trae IP, dispositivo, medio de pago ni tipo de orden, el generador del proyecto agrega esos campos y casos sintéticos de fraude, duplicados, eventos inválidos, órdenes express y programadas, y paquetes de productos que se compran juntos. La etiqueta es_fraude del generador solo se usa para medir; ninguna etapa la usa para decidir.

El orquestador (src/pipeline.py) pasa cada lote del stream por las etapas 0 → 1 → 6 → 2 → 3 → 4 → 8. La detección de fraude va antes que la prioridad para que la cola use su decisión. Al cerrar el stream se construyen las etapas 5, 7 y 9 sobre el catálogo completo. El diagrama del flujo está en docs/diagrama_pipeline.html y el pipeline se ejecuta con run_pipeline.py, con el notebook pipeline_completo.ipynb o desde el panel de Streamlit.

Etapa	Estructura propia	Comparada con	Responsable
0 Ingesta	Cola FIFO (lista enlazada)	deque, buffer circular, list.pop(0)	Axcel Fuentes
1 Preprocesamiento	Merge sort y búsqueda binaria	sorted(), quicksort, inserción	Axcel Fuentes
6 Fraude	Filtro de Bloom y Count-Min Sketch	set y dict exactos	Axcel Fuentes
2 Prioridad	MinHeap	heapq, lista ordenada	Carlos Fukumoto
3 Lookup	Tabla hash, caché LRU, matriz dispersa	dict, OrderedDict	Carlos Fukumoto
4 Índice	B+Tree de orden 64	bisect, recorrido lineal	Carlos Fukumoto
8 Grafo	PageRank, Louvain	Grado, Label Propagation	Carlos Fukumoto
5 Multidimensional	KD-Tree	KNN exacto	Sebastián Gallegos
7 Similitud	IVF, MinHash + LSH	KNN y Jaccard exactos	Sebastián Gallegos
9 Servicio	Top-K con heap, Trie	Ordenar todo el catálogo	Sebastián Gallegos

Método de medición: semilla 42, recolector de basura apagado, perf_counter para el tiempo y tracemalloc para la memoria pico, en una corrida aparte. Se usan hasta 100 repeticiones por tamaño y entre 3 y 10 con 1M; las variantes cuadráticas se cortan en 10K o 100K. Para comparar con la teoría se usa la pendiente en escala log-log: 1 es lineal, cerca de 1,1 es n log n y 2 es cuadrático. Las estructuras de la biblioteca estándar están escritas en C y son más rápidas; lo que se compara es la forma de las curvas. Las etapas 0, 1 y 6 se midieron en una laptop con Intel Core Ultra 7 165U en batería, y las etapas 2, 3, 4 y 8 en una con Intel Core i7-13620H enchufada, ambas con Python 3.13.

2. Etapas 0 y 1: ingesta y preprocesamiento

La etapa 0 es una cola FIFO sobre una lista enlazada con punteros a la cabeza y a la cola: encolar y desencolar cuestan O(1) porque nunca se recorre la lista, y entrega lotes de 1.000 eventos. Si el buffer se llena, encolar devuelve False y se entrega un lote, sin perder eventos. Se eligió la lista enlazada porque crece sin fijar una capacidad; el buffer circular gasta menos memoria, pero exige fijarla.

La etapa 1 limpia cada lote en O(n log n). Valida campos, fechas, tipos, ids y precios en O(n) y descarta los inválidos. Ordena por hora con merge sort iterativo, que es estable y garantiza O(n log n) aunque el orden de llegada se rompa. La clave (hora, sesión, producto, tipo, usuario, precio) deja las copias contiguas, así que deduplicar

es una sola pasada. La búsqueda binaria ubica en O(log n) dónde empieza una ventana de tiempo (Cormen et al., 2022).

Operación	Tiempo	Espacio
Encolar, desencolar	O(1)	O(1) por evento
Validar y normalizar un lote	O(n)	O(n)
Merge sort	O(n log n)	O(n)
Deduplicar por vecinos	O(n)	O(n)
Búsqueda binaria	O(log n)	O(1)

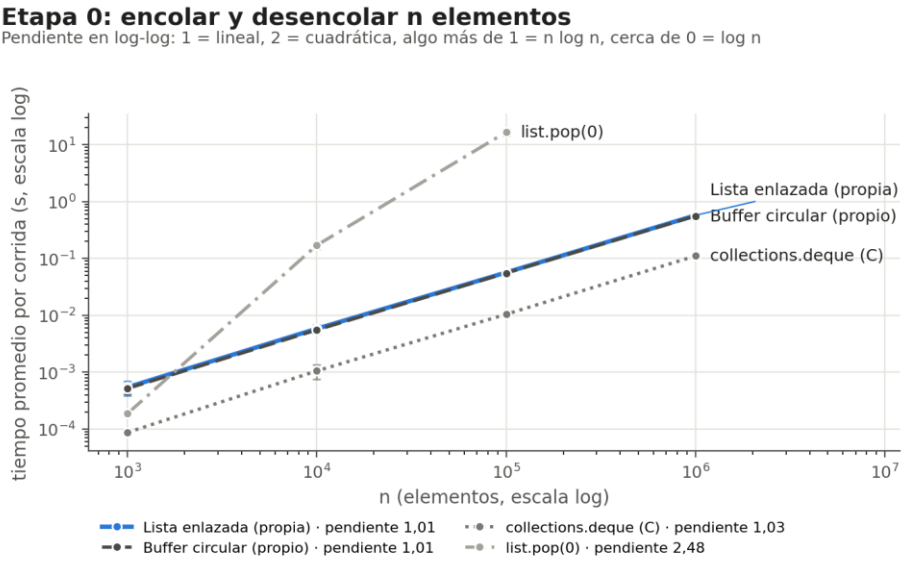


Figura 1. Tiempo de encolar y desencolar n elementos en escala log-log: lista enlazada, buffer circular, deque y list.pop(0).

- **Cola.** La lista enlazada crece de forma lineal (pendiente 1,01); list.pop(0) es cuadrática (2,48) y con 100K ya es 295 veces más lenta. A cambio, la lista enlazada usa 48 MB con 1M frente a 8 MB del buffer circular, por el objeto que crea para cada evento.
- **Ordenamiento.** Merge sort mide 1,21 con entrada aleatoria y la inserción 2,03. Con la entrada casi ordenada del stream, la inserción es más rápida (1,60 s frente a 4,42 s con 1M), pero con entrada aleatoria ya tarda 5,7 s con 10K. Quicksort fue más lento que merge sort por los intercambios en Python.

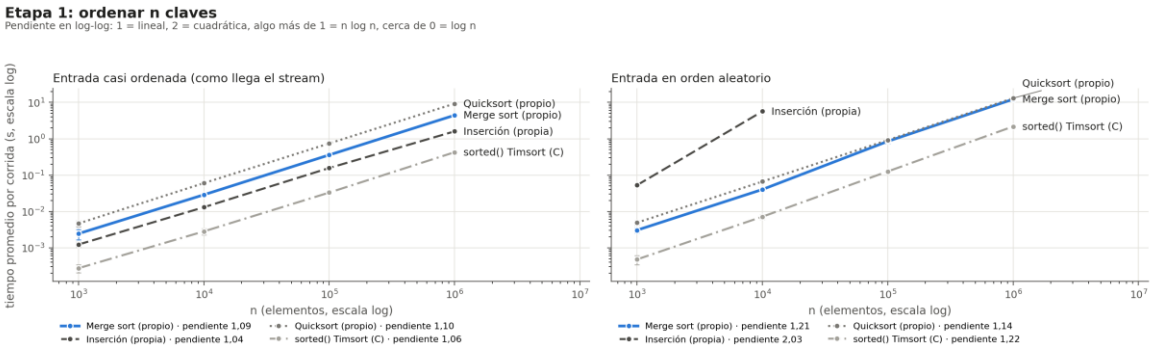


Figura 2. Tiempo de ordenar n claves en escala log-log, con entrada casi ordenada (izquierda) y aleatoria (derecha): merge sort, quicksort, inserción y sorted().

- **Deduplicación.** Deduplicar por vecinos tras ordenar es algo más rápido que un conjunto hash (0,77 s frente a 0,87 s con 1M), usa 3,3 veces menos memoria y deja el lote ordenado.

Operación con 1M de claves	Tiempo	Memoria pico
Deduplicar con merge sort propio y vecinos	4,81 s	39,6 MB
Deduplicar con sorted() y vecinos	0,77 s	16,9 MB
Deduplicar con un conjunto hash	0,87 s	55,9 MB
100 búsquedas binarias propias	0,41 ms	< 0,1 MB
100 búsquedas con bisect (C)	0,12 ms	< 0,1 MB

- **De punta a punta.** Con 1M de eventos en lotes de 1.000, se descartaron 9.557 inválidos y 46.681 duplicados. La deduplicación tuvo 100 % de precisión y 99,7 % de recall: solo se escapan las copias que caen en otro lote.

3. Etapa 6: detección de fraude

La etapa 6 marca cada evento con una decisión de fraude, un puntaje y sus motivos, sin detener el flujo. En un stream de pagos no conviene guardar todo de forma exacta, así que usa dos estructuras probabilísticas que ceden un poco de exactitud a cambio de mucha memoria.

- **Filtro de Bloom.** Guarda la lista negra de IP y dispositivos en un arreglo de m bits con k funciones hash (Bloom, 1970). Al consultar, si algún bit está en 0 la clave no está; si todos están en 1, probablemente sí. No tiene falsos negativos. Con $m = -n \ln p / (\ln 2)^2$ y $k = (m/n) \ln 2$, un objetivo de 1 % da unos 9,6 bits y 7 hashes por clave, calculados por doble hashing desde un solo resumen.

- **Count-Min Sketch.** Cuenta eventos por IP y dispositivo en ventanas de 10 segundos con una tabla fija de d filas por w columnas; la estimación es el mínimo de las d filas, así que nunca subestima una ráfaga (Cormode y Muthukrishnan, 2005). Con ε = 0,001 y δ = 0,01 tiene 5 filas y 2.719 columnas.
- **Reglas.** Un evento es fraude si su IP o dispositivo está en la lista negra, si llega a 8 eventos en la ventana, en cuyo caso se marca la ráfaga completa, o si ese identificador ya superó el umbral antes.

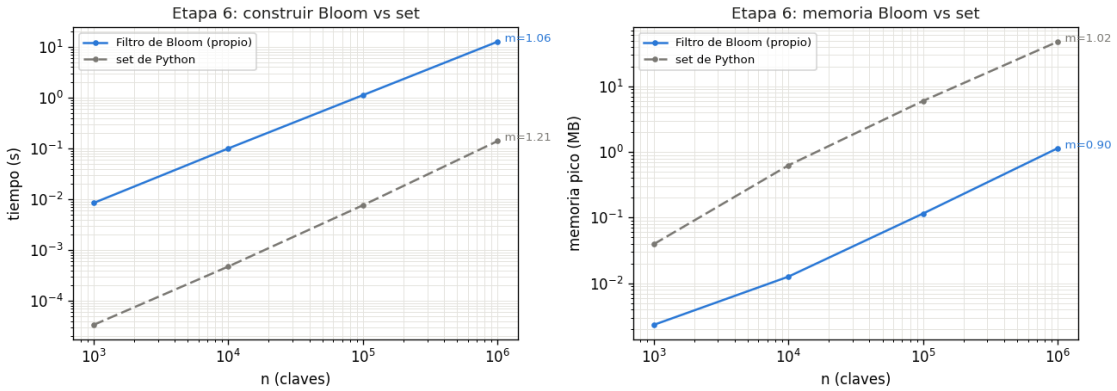


Figura 3. Tiempo de construcción (izquierda) y memoria pico (derecha) del filtro de Bloom y del set de Python, en escala log-log.

Medida con 1M de claves	**Estructura propia**	**Alternativa exacta**
Memoria de la lista negra	Bloom: 1,14 MB	set: 48,0 MB
Falsos positivos (objetivo 1 %)	0,99 %	0 %
Memoria del conteo por ventana	Count-Min: 0,13 MB fija	dict: 44,0 MB
Error relativo medio del conteo	0,035 %, nunca subestima	0 %
Detector completo	14.334 eventos/s	—

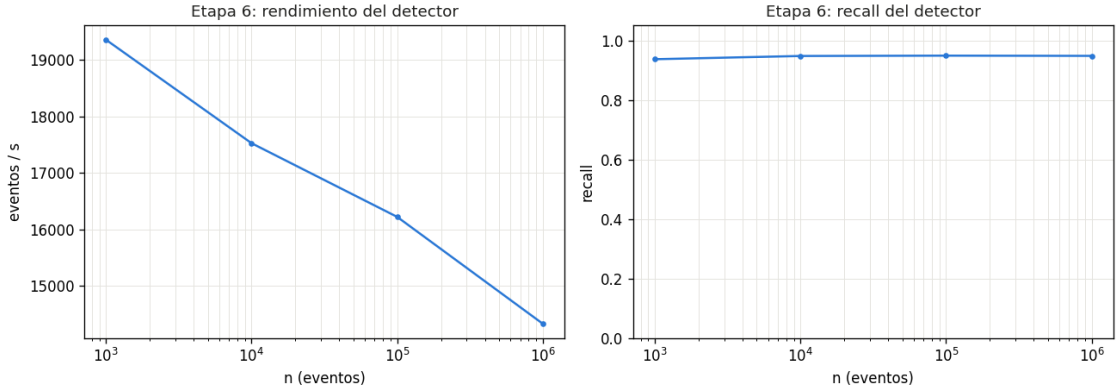


Figura 4. Rendimiento (izquierda) y recall (derecha) del detector de fraude según la cantidad de eventos, antes de marcar la ráfaga completa.

El Bloom ocupa 42 veces menos que el set y el Count-Min unas 340 veces menos que el dict, a cambio de ser entre 15 y 90 veces más lentos por estar en Python. El detector mantuvo precisión 1,0 de 1K a 1M eventos. Su recall medido fue 0,95, antes de agregar la marca de la ráfaga completa; en el pipeline integrado actual, con 10.000 eventos, marcó 613 eventos con precisión y recall de 1,0. Los límites son la ventana fija, que cuenta un evento atrasado en la ventana vigente, y un filtro de Bloom que no permite borrar claves.

4. Etapa 2: cola con prioridades

La etapa 2 entrega cada lote en orden de urgencia con un MinHeap propio guardado en una lista: el padre de la posición i está en (i − 1) // 2, el menor elemento siempre está en la posición 0, y push y pop recorren a lo sumo la altura del árbol (Cormen et al., 2022). La clave es (prioridad, hora, contador, evento): a igual prioridad sale el más antiguo, y el contador evita comparar dos eventos entre sí. La prioridad 0 sale de la decisión de la etapa 6, nunca de la etiqueta del generador.

Prioridad	Nivel	Cuándo se asigna
0	Fraude o pago	La etapa 6 lo marcó, aunque sea una orden express
1	Express	Orden con entrega express
2	Estándar	Compra sin tipo de orden
3	Programada	Orden con entrega programada
4	Navegación	Vistas, carritos y retiros del carrito

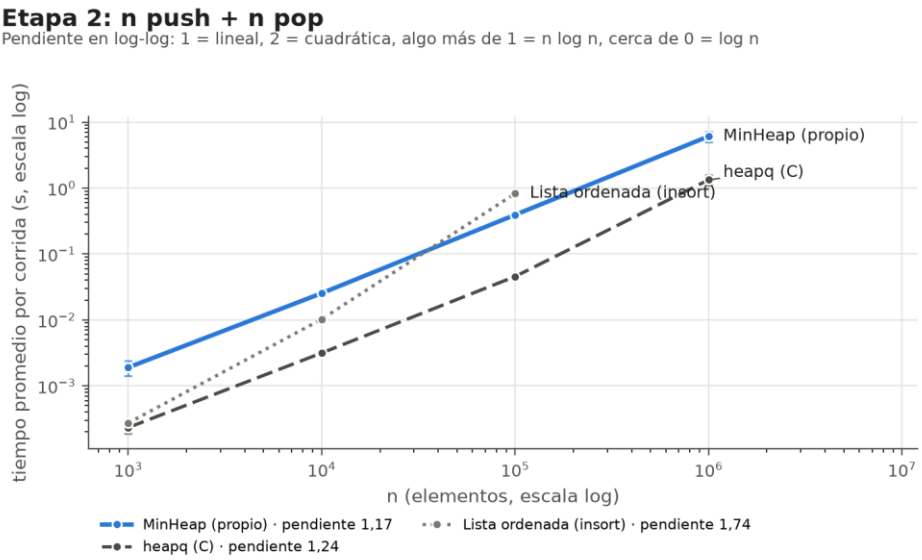


Figura 5. Tiempo de n push + n pop en escala log-log: MinHeap propio, heapq y lista ordenada.

El MinHeap mide 1,17 y heapq 1,24, ambos n log n; heapq es 4,6 veces más rápido por estar en C. La lista ordenada, que inserta en O(n), mide 1,74 y con 100K ya es 2,1 veces más lenta. Construir con heapify cuesta O(n) (pendiente 1,05) y es 28 % más rápido que n push. Con 1M de eventos, cada operación hace como máximo unos 20 intercambios y la etapa completa procesa 6 µs por evento.

5. Etapas 3 y 4: lookup, caché e índice

- **Tabla hash.** Guarda una ficha por producto (categoría, marca, precio y contadores) con encadenamiento. El índice de cubeta se calcula con hashing multiplicativo de Fibonacci: se multiplica el id por $2^{64}/\varphi$ y se toman los bits altos, lo que reparte bien ids consecutivos (Knuth, 1998). Al superar un factor de carga de 0,75 duplica las cubetas y reubica todo: O(n) esa vez, O(1) amortizado.
- **Caché LRU.** Tabla hash más lista doblemente enlazada con centinelas: consultar mueve el nodo al frente y guardar expulsa el del final, todo en O(1). Cuando un evento cambia una ficha, su copia en caché se invalida.
- **Matriz dispersa.** Diccionario de claves usuario × producto con doble índice, por fila y por columna. Solo guarda celdas no nulas: una matriz de interacciones ponderadas (vista 1, carrito 2, compra 4) y otra de compras, que entrega a la etapa 7 los compradores de cada producto.
- **B+Tree.** Índices por hora y por precio, de orden 64. Los nodos internos solo guían y las claves viven en hojas enlazadas: una consulta por rango baja una vez y recorre las hojas sin volver a subir (Bayer y McCreight, 1972). Cuando cambia un precio, la entrada vieja se borra de forma perezosa.

Operación	Tiempo	Espacio
Tabla hash: insertar, buscar	O(1) promedio	O(n)
Caché LRU: obtener, guardar	O(1)	O(capacidad)
Matriz dispersa: sumar, leer	O(1) promedio	O(celdas no nulas)
B+Tree: insertar, buscar	O(log n)	O(n)
B+Tree: rango con k resultados	O(log n + k)	O(k)

Medida con 1M	Propia	Alternativa	Pendiente
Tabla hash: inserción y búsqueda	5,06 s · 303 MB	dict: 0,33 s · 63 MB	1,19
Caché LRU: n accesos	1,32 s	OrderedDict: 0,14 s	0,98
Matriz dispersa: n interacciones	2,19 s · 119 MB	Densa: 40 GB	1,19
B+Tree: n inserciones	3,19 s	Orden 16: 4,07 s	1,22
B+Tree: 100 rangos	4,6 ms	Recorrido lineal: 4,79 s	0,15

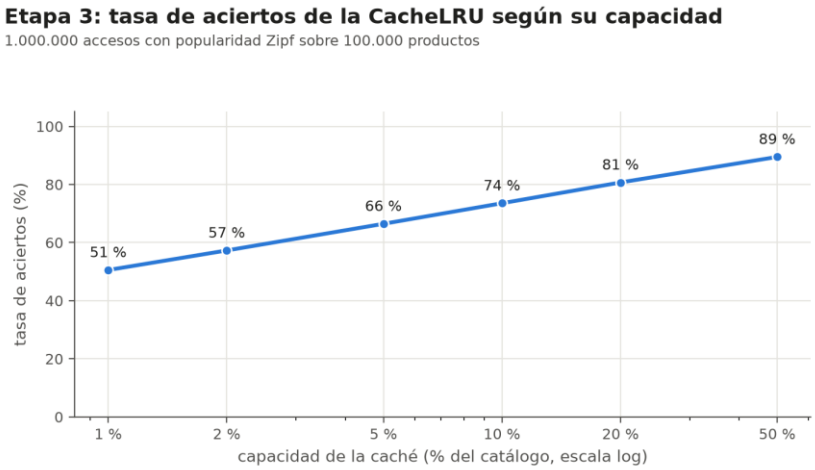


Figura 6. Tasa de aciertos de la caché LRU según su capacidad, con 1M de accesos de popularidad Zipf sobre 100K productos.

- **Hashing.** Con 1M de claves la tabla usa 2^{21} cubetas, carga 0,48 y su cadena más larga tiene 2 elementos: el hashing de Fibonacci reparte bien. Usa 4,8 veces más memoria que dict porque cada par es un objeto de Python.
- **Tamaño de la caché.** Con 100K productos y popularidad Zipf, una caché del 1 % del catálogo acierta el 50,6 % de las consultas; del 10 %, el 73,5 %; y del 50 %, el 89,5 %. Pocos productos concentran las visitas.

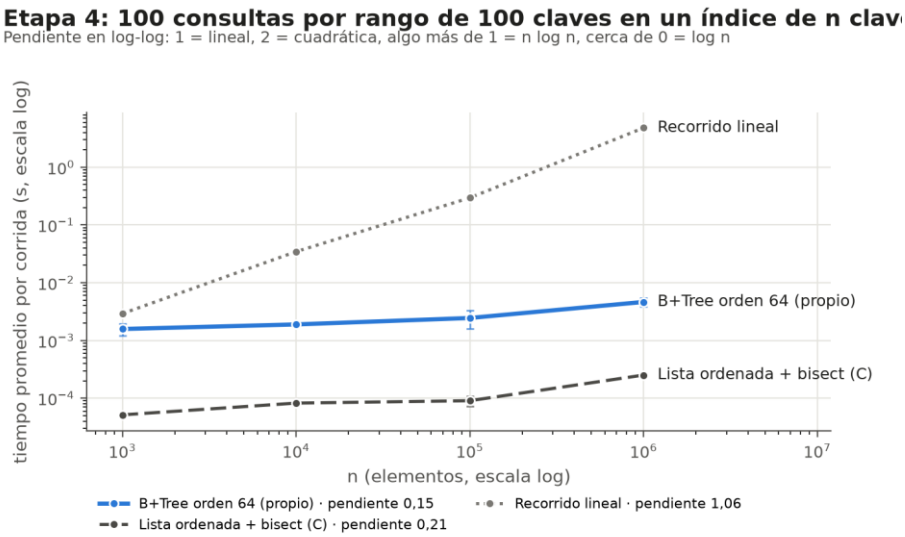


Figura 7. Tiempo de 100 consultas por rango en un índice de n claves, en escala log-log: B+Tree de orden 64, lista ordenada con bisect y recorrido lineal.

- **Rangos.** El B+Tree responde mil veces más rápido que el recorrido lineal y su altura es 4 con 1M. bisect consulta más rápido, pero una lista ordenada inserta en O(n) y no sirve para un stream que no deja de insertar.

6. Etapas 5 y 7: vecinos y similitud

La etapa 5 representa cada producto con un vector de precio mediano normalizado, popularidad ponderada por tipo de evento y categoría en one-hot, y lo indexa en un KD-Tree que alterna la dimensión de corte por la mediana y poda las ramas que no pueden contener un vecino más cercano (Bentley, 1975). Construirlo cuesta O(n log² n) en esta versión, y una consulta suele revisar una fracción de los puntos, aunque en el peor caso recorre O(n). El KNN exacto por fuerza bruta, O(nd + n log n), sirve de referencia.

La etapa 7 ofrece dos métodos aproximados. IVF agrupa los vectores en 16 listas invertidas por centroide y solo revisa las listas más cercanas a la consulta (Johnson et al., 2017). MinHash resume los compradores de cada producto en una firma que aproxima la similitud de Jaccard, y LSH divide la firma en bandas para comparar solo candidatos, sin revisar todos los pares (Broder, 1997; Indyk y Motwani, 1998).

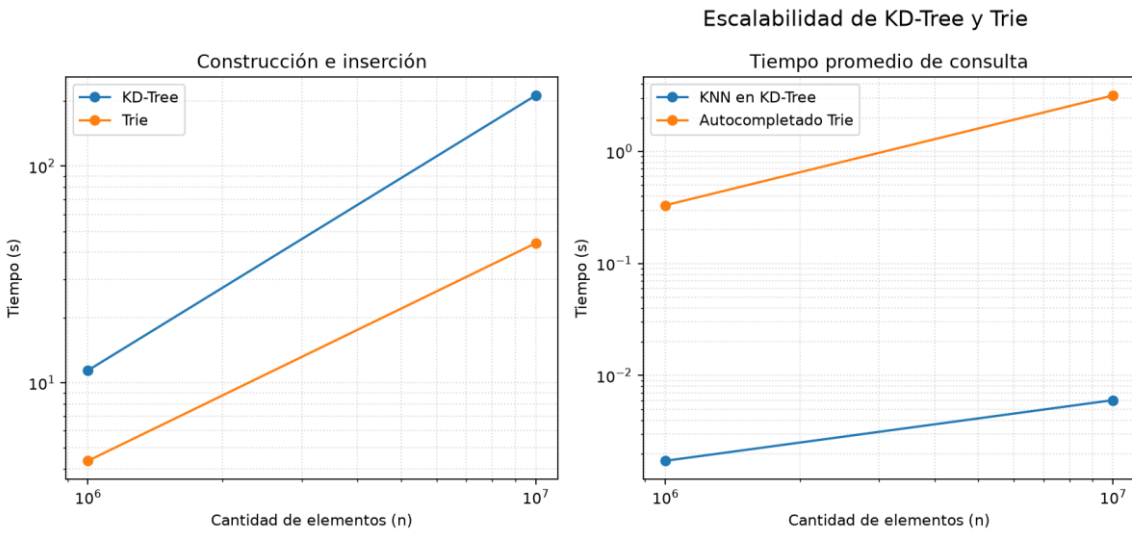


Figura 8. Tiempo de construcción y de consulta del KD-Tree y del Trie en escala log-log, de 1M a 10M elementos.

Medida	**1K**	**10K**	**100K**
MinHash + LSH: construcción	1,41 s	17,9 s	318 s
MinHash + LSH: consulta	0,17 ms	1,53 ms	17,6 ms
MinHash + LSH: recall@10	0,78	0,88	0,94
MinHash + LSH: memoria pico	12 MB	112 MB	1.007 MB

Con 1M de vectores, construir el KD-Tree tomó 11,4 s y cada consulta 1,7 ms; con 10M, 212 s y 6,0 ms: la consulta crece mucho más lento que n. El recall de MinHash sube con el tamaño porque hay más compradores en común. En el pipeline integrado con 10.000 eventos, el KD-Tree e IVF recuperaron los 5 vecinos exactos en todas las consultas y MinHash + LSH el 87 %. La escalabilidad de IVF queda por medir.

7. Etapa 8: grafo de co-compra

Los nodos son productos y una arista une dos productos comprados en la misma sesión, porque REES46 no trae un id de orden; el peso cuenta en cuántas sesiones ocurrió. El grafo se guarda como lista de adyacencia y se arma con cada lote: una orden de k productos agrega k(k − 1)/2 aristas. PageRank mide la influencia como la probabilidad estacionaria de un paseo aleatorio que sigue las aristas con probabilidad d = 0,85 y salta a un producto cualquiera con 1 − d (Brin y Page, 1998):

$$PR(v) = (1 - d) / N + d \cdot \Sigma\, PR(u) \cdot w(u, v) / W(u)$$

Se calcula con el método de las potencias hasta un cambio menor que 10^{-10}, a O(N + E) por iteración, y el Top-K usa el MinHeap de la etapa 2. Las comunidades se buscan con Louvain, que mueve cada nodo a la comunidad vecina que más aumenta la

modularidad Q y luego colapsa cada comunidad en un nodo (Newman y Girvan, 2004; Blondel et al., 2008), y con Label Propagation, donde cada nodo copia la etiqueta de más peso entre sus vecinos (Raghavan et al., 2007). Para saber si aciertan se usa un grafo sintético con comunidades plantadas y la información mutua normalizada (NMI).

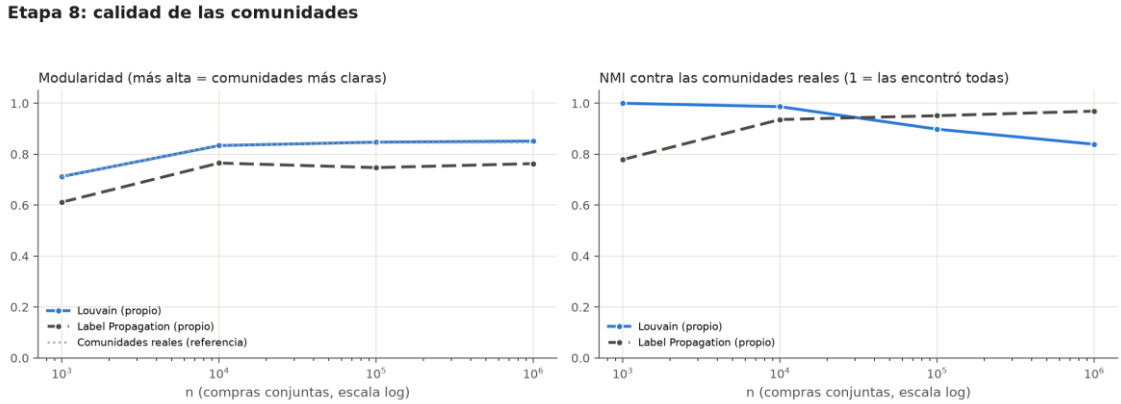


Figura 9. Modularidad (izquierda) y NMI contra las comunidades reales (derecha) de Louvain y Label Propagation, según la cantidad de compras conjuntas.

Medida con 1M de pares	Louvain	Label Propagation	Reales
Modularidad	0,851	0,763	0,846
Comunidades	493	12.482	4.981
NMI contra las reales	0,839	0,969	1
Tiempo · memoria pico	62 s · 149 MB	51 s · 32 MB	—

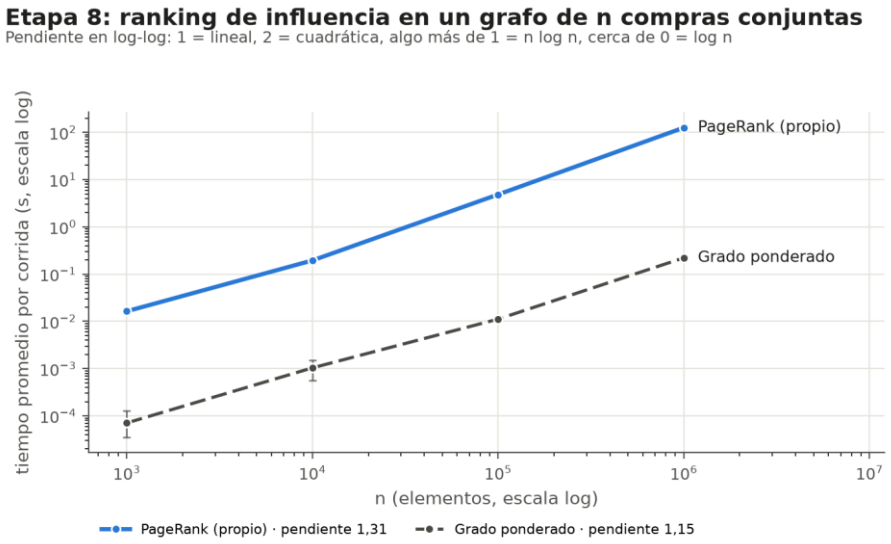


Figura 10. Tiempo de PageRank y del ranking por grado ponderado en escala log-log, según la cantidad de compras conjuntas.

- **PageRank frente a grado.** Con 1M de pares (199.954 productos y 891.567 aristas), PageRank converge en 73 iteraciones y tarda 126 s, frente a 0,22 s del grado. Su correlación de Spearman con el grado es 0,95, pero solo 5 de los 10 primeros coinciden: importa con quién se compra un producto, no solo cuántas veces.
- **Louvain frente a Label Propagation.** Louvain logra más modularidad que las comunidades reales, pero acierta menos: encuentra 493 grupos de 4.981 porque une los pequeños, el límite de resolución de la modularidad (Fortunato y Barthélemy, 2007). Label Propagation fragmenta y es menos estable entre semillas. Se eligió Louvain porque el servicio necesita grupos amplios y estables.

Estabilidad con 10 semillas	**Louvain**	**Label Propagation**
1K pares: modularidad media	0,712 ± 0,000	0,668 ± 0,015
10K pares: modularidad media	0,833 ± 0,001	0,755 ± 0,021
100K pares: modularidad media	0,847 ± 0,000	0,762 ± 0,006
100K pares: comunidades	172 ± 5	1.293 ± 59

- **Validación.** El PageRank propio coincide con el de NetworkX hasta $6 \cdot 10^{-11}$, con el mismo Top-10, y el Louvain propio logra la misma modularidad (Hagberg et al., 2008).

8. Etapa 9: servicio y pruebas de punta a punta

El servicio reúne las salidas de todas las etapas en un panel de Streamlit: recomendaciones por similitud, productos con compradores en común, ranking de influencia con sus comunidades, alertas de fraude, la cola con prioridades y un panel de ventas que no cuenta las compras retenidas por fraude. El Top-K reutiliza el MinHeap de la etapa 2 en O(n log k), y el autocompletado de marcas y categorías usa un Trie que inserta una palabra de largo L en O(L) (Knuth, 1998); con 10M de palabras su construcción tomó 44 s, de forma lineal.

El repositorio tiene 229 pruebas automáticas, entre ellas una que pasa 1.000 eventos por las 10 etapas y exige salida en cada una. Con datos reales de REES46 se corrió el pipeline completo sobre 100.000 eventos del 1 de octubre de 2019:

Etapa (100.000 eventos reales)	Tiempo	Resultado
0 y 1: ingesta y limpieza	2,6 s	17 duplicados eliminados
6: fraude (sintético agregado)	4,1 s	9.589 eventos marcados
2, 3 y 4: prioridad, lookup e índice	1,8 s	20.621 productos; B+Tree de altura 3
8: grafo	0,03 s	843 productos y 175 aristas
7: similitud (IVF y MinHash)	126,7 s	IVF y KD-Tree con recall@5 de 1,0

El pipeline completo tomó 138 s con 258 MB de memoria pico. La etapa 7 domina porque construye sus índices sobre todo el catálogo real, mucho más grande que el sintético. El grafo real resultó disperso: pocas sesiones compran más de un producto, así que hacen falta más eventos para que aparezcan comunidades.

9. Conclusiones y límites

- **La teoría se cumple.** Todas las estructuras propias crecen como su complejidad: lineales las de O(1) por operación, n log n el heap, el ordenamiento y el B+Tree, y casi constantes las consultas por rango. Las alternativas cuadráticas se separan desde 10K.
- **Memoria frente a exactitud.** El filtro de Bloom, el Count-Min y la matriz dispersa reducen la memoria entre 40 y 300 veces con errores acotados, que es lo que permite procesar un stream sin guardarlo todo.
- **Cada pregunta, su estructura.** La tabla hash no guarda orden, por eso los rangos van al B+Tree; el heap solo da el más urgente, que es lo que necesita la cola; y una modularidad alta no garantiza comunidades correctas.
- **Límites.** Las mediciones de 10M quedan pendientes para la mayoría de las estructuras; el fraude y los tipos de orden son sintéticos; la sesión se usa como orden; y las compras marcadas como fraude todavía entran al grafo y a la matriz de compras.
- **Trabajo futuro.** Filtrar el fraude antes del grafo, medir IVF y las estructuras restantes en 10M, y probar Leiden para comunidades más finas en grafos grandes.

Referencias

Bayer, R., & McCreight, E. (1972). Organization and maintenance of large ordered indexes. Acta Informatica, 1(3), 173–189. https://doi.org/10.1007/BF00288683

Bentley, J. L. (1975). Multidimensional binary search trees used for associative searching. Communications of the ACM, 18(9), 509–517. https://doi.org/10.1145/361002.361007

Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. Journal of Statistical Mechanics, 2008(10), P10008. https://doi.org/10.1088/1742-5468/2008/10/P10008

Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors. Communications of the ACM, 13(7), 422–426. https://doi.org/10.1145/362686.362692

Brin, S., & Page, L. (1998). The anatomy of a large-scale hypertextual Web search engine. Computer Networks and ISDN Systems, 30(1–7), 107–117. https://doi.org/10.1016/S0169-7552(98)00110-X

Broder, A. Z. (1997). On the resemblance and containment of documents. En Compression and Complexity of Sequences 1997 (pp. 21–29). IEEE. https://doi.org/10.1109/SEQUEN.1997.666900

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Cormode, G., & Muthukrishnan, S. (2005). An improved data stream summary: The count-min sketch and its applications. Journal of Algorithms, 55(1), 58–75. https://doi.org/10.1016/j.jalgor.2003.12.001

Fortunato, S., & Barthélemy, M. (2007). Resolution limit in community detection. PNAS, 104(1), 36–41. https://doi.org/10.1073/pnas.0605965104

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. En Proceedings of the 7th Python in Science Conference (pp. 11–15).

Indyk, P., & Motwani, R. (1998). Approximate nearest neighbors: Towards removing the curse of dimensionality. En Proceedings of the 30th ACM STOC (pp. 604–613). https://doi.org/10.1145/276698.276876

Johnson, J., Douze, M., & Jégou, H. (2017). Billion-scale similarity search with GPUs. arXiv. https://doi.org/10.48550/arXiv.1702.08734

Knuth, D. E. (1998). The art of computer programming: Vol. 3. Sorting and searching (2nd ed.). Addison-Wesley.

mkechinov. (s. f.). eCommerce behavior data from multi category store [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store

Newman, M. E. J., & Girvan, M. (2004). Finding and evaluating community structure in networks. Physical Review E, 69(2), 026113. https://doi.org/10.1103/PhysRevE.69.026113

Raghavan, U. N., Albert, R., & Kumara, S. (2007). Near linear time algorithm to detect community structures in large-scale networks. Physical Review E, 76(3), 036106. https://doi.org/10.1103/PhysRevE.76.036106